Nombres: Arturo Sebastian Pérez Zapata Cesar Maximiliano Peña Mendoza Diego Alberto Sierra Mendoza	**Matrícula**: 2142942 2144135 2143733
Nombre del curso: Componentes Lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: 3 Diseño Arquitectural del Sistema Nova Rent
Fecha: 15/10/2026	

ACT-F03 · Diseño Arquitectural del Sistema

NovaRent

1. Elegir patrones de lógica de negocio

1.1 Reglas del caso que condicionan la decisión

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Procesar devolución y cargos tardíos	Agente de sucursal	Tolerancia de 29 min, cobro por hora con tope de 1 día.	Devolución y cargo por retraso registrados.
CU-02	Cobrar kilometraje y combustible	Agente / Sistema	200 km/día incluidos ($3 MXN/km extra); sin cargo si es ≤ 1/8 de tanque.	Cargos aplicados a la liquidación.
CU-03	Extender periodo de renta	Agente / Admin.	Agente aprueba ≤ 48 h; más de 48 h requiere Admin.	Renta extendida o enviada a aprobación.
CU-04	Cancelar reserva y liberar no-show	Cliente / Sistema	Cancelación gratis ≥ 24 h; no-show se libera a las 2 h.	Reserva liberada y flota actualizada.

Conclusión: muchas reglas, con excepciones, permisos por rol y políticas que cambian.

1.2 Patrones evaluados

Patrón	Dónde vive la lógica	Funciona bien cuando	En NovaRent
Transaction Script	Un procedimiento por caso de uso	Reglas simples, pocos casos de uso	Devolución, extensión y upgrade repetirían las mismas reglas
Table Module	Una clase por tabla, sobre conjuntos de registros	Datos tabulares, reglas moderadas	Las reglas cruzan contrato, inspección y cargos; queda dispersa
Active Record	Cada registro se guarda y lleva su lógica	CRUD con reglas ligeras	Mezcla reglas con persistencia; difícil de probar sin BD
Domain Model	Objetos con datos y comportamiento (Aggregates, Value Objects, Policies)	Reglas ricas, cambiantes y con excepciones	Encaja con tolerancias, topes, permisos y excepciones
Service Layer (complemento)	Fachada de casos de uso que coordina	Varios clientes: UI, API, jobs	Coordina; no decide

1.3 Tabla comparativa (1 = pobre, 5 = excelente)

Criterio	Transaction Script	Table Module	Active Record	Domain Model + Service Layer
Reglas complejas con excepciones (upgrade, extensión, disputa)	1	2	3	5
Cambio frecuente de políticas (tolerancias, tarifas)	2	2	3	5
Permisos y límites por rol (agente ≤ 48 h, Administración > 48 h)	2	2	2	5
Pruebas sin base de datos	3	3	2	5
Lenguaje ubicuo reflejado en el código	1	2	3	5
Bajo acoplamiento a la BD	3	2	1	5
Esfuerzo inicial y curva de aprendizaje	5	4	4	2
Total (máx. 35)	17	17	18	32

1.4 Decisión

Domain Model + Service Layer.

- Las reglas viven en un solo lugar (Aggregates y Policies), no en cada flujo.
- Las políticas cambian por configuración: 29 min, 200 km, 1/8, 48 h.
- Cada regla se prueba sin base de datos.
- El código usa los términos del negocio (ver sección 2.7).
- El Service Layer solo coordina: transacción, repositorios y eventos.

1.5 Costos aceptados

- Más esfuerzo inicial y curva de aprendizaje del equipo.
- Mapeo objeto-relacional para persistir los Aggregates.
- Consistencia eventual entre Aggregates (se resuelve con Domain Events).

1.6 Excepción

Los reportes de ocupación, ingresos y rotación (US19) no pasan por el Domain Model. Usan consultas directas, estilo Transaction Script. Referencias: ADR-001 y ADR-002.

2. Mapa de dominio

2.1 Contexto delimitado y subdominios

Un solo Bounded Context: **Operación de Renta**.

- **Core:** Reservación, Contrato, Inspección, Liquidación.
- **Soporte:** Flota, Limpieza, Catálogo (Categoría, Sucursal, Empleado).
- **Genérico:** Pagos y Notificaciones (puertos externos).

2.2 Aggregates

Aggregate (Root)	Entidades internas	Value Objects	Invariantes clave	Refiere por ID
Reservacion	—	PeriodoRenta	Garantiza categoría, no unidad. Cancelación sin penalización con ≥ 24 h. Se libera a las 2 h de no-show. Upgrade solo a la categoría inmediata superior.	ClienteId, CategoriaId, SucursalId
ContratoRenta	Extension, ConductorAdicional	PeriodoRenta, CoberturaSeguro, Kilometraje, CondicionesContrato, ReferenciaPago	Un contrato = una unidad. Extensión ≥ 2 h antes del vencimiento; el agente ≤ 48 h, más requiere Administración.	Folio, ClienteId, VehiculoId, CategoriaId
InspeccionVehiculo	—	NivelCombustible, Kilometraje, RegistroDanio, EvidenciaFoto, FirmaDigital, DiferenciasInspeccion	Toda foto lleva sello de tiempo. Firmada = inmutable. La entrega exige firma del cliente.	ContratoId, VehiculoId, EmpleadoId
Liquidacion	Cargo, Deposito	Dinero, Disputa	Sin cargo por combustible si la diferencia es ≤ 1/8. El monto de daño lo fija Administración. El depósito se libera a 5 días hábiles sin incidencias.	ContratoId
Vehiculo	—	Kilometraje	Solo un vehículo Disponible se asigna. Al devolverse pasa a EnLimpieza.	CategoriaId, SucursalId
TareaLimpieza	—	—	Una tarea abierta por devolución.	VehiculoId, EmpleadoId
Cliente	—	Licencia, MetodoPagoRef	Licencia vigente y tarjeta válida. Edad mínima 25 (menor: cargo por conductor joven).	—
CategoriaVehiculo	—	Dinero, Kilometraje	El nivel define cuál es la categoría superior.	—
Sucursal	—	—	—	—
Empleado	—	—	El rol determina las autorizaciones.	SucursalId

2.3 Entidades y Value Objects

- **Entidad:** tiene identidad y ciclo de vida (Extension, ConductorAdicional, Cargo, Deposito).
- **Value Object:** sin identidad, inmutable, se compara por valor.

Value Object	Contenido	Regla
PeriodoRenta	inicio, fin	fin > inicio
Dinero	monto, moneda (MXN)	sin montos negativos
Kilometraje	Valor	≥ 0
NivelCombustible	octavos (0 a 8)	0 ≤ octavos ≤ 8

Value Object	Contenido	Regla
Licencia	número, vigencia	vigente en la fecha de renta
MetodoPagoRef	token, últimos 4	nunca guarda el número de tarjeta
ReferenciaPago	id externo	referencia del depósito retenido
EvidenciaFoto	uri, tomadaEn	sello de tiempo obligatorio
RegistroDanio	zona, descripción, fotos	al menos una foto
FirmaDigital	firmante, firmadaEn	—
Disputa	motivo, fecha, resolución, resueltaPor	la resuelve solo Administración
CoberturaSeguro	tipo, costo diario	—
DiferenciasInspeccion	km recorridos, octavos faltantes, daños nuevos	resultado de comparar entrega vs devolución
CondicionesContrato	periodo pactado, devolución real, km incluido, categoría	datos que usa Liquidacion para cobrar

2.4 Domain Services / Policies

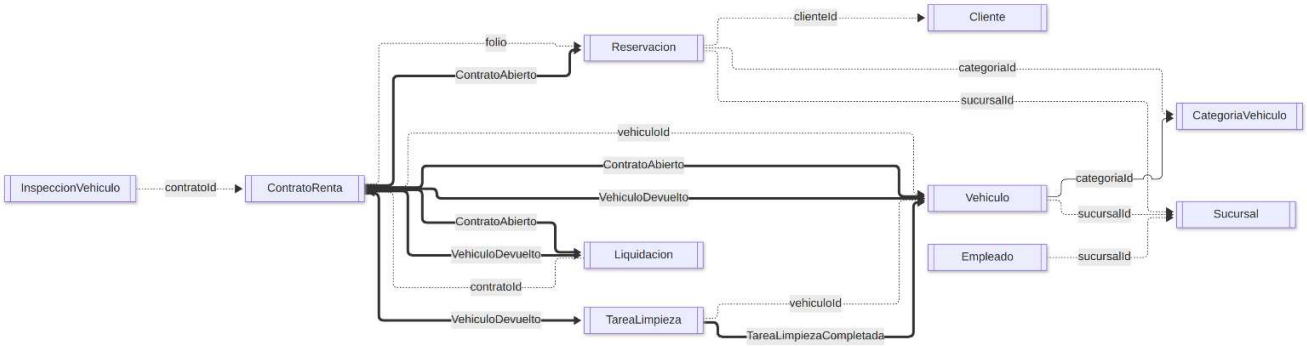
Servicio	Responsabilidad	Parámetros
PoliticaCancelacion	Penalización y no-show	24 h, 2 h
PoliticaCargos	Retraso, km excedente, combustible	29 min, 200 km/día, $3 MXN/km, 1/8; tarifa por octavo: por definir
PoliticaExtension	Quién autoriza una extensión	48 h, 2 h de anticipación
PoliticaUpgrade	Categoría superior de una categoría	nivel de categoría
ServicioDisponibilidad	Hay unidad y asignación	categoría, sucursal, periodo

2.5 Domain Events

Evento	Lo emite	Reacciona
`ReservacionCreada`	Reservacion	Notificaciones
`ReservacionCancelada`	Reservacion	Notificaciones, disponibilidad
`ReservacionLiberadaPorNoShow`	Reservacion	Disponibilidad
`UpgradeOfrecido`	Reservacion	Interfaz del agente
`UpgradeRechazado`	Reservacion	Lista de espera o cancelación
`InspeccionEntregaFirmada`	InspeccionVehiculo	Habilita abrir el contrato
`ContratoAbierto`	ContratoRenta	Reservacion → Convertida; Vehiculo → Rentado; abre Liquidacion con depósito
`ExtensionRequiereAutorizacion`	ContratoRenta	Bandeja de Administración
`RentaExtendida`	ContratoRenta	Disponibilidad, Notificaciones
`InspeccionDevolucionCompletada`	InspeccionVehiculo	Notificaciones
`VehiculoDevuelto`	ContratoRenta	Liquidacion calcula cargos; Vehiculo → EnLimpieza; crea TareaLimpieza
`CargosCalculados`	Liquidacion	Agente y Administración (daños por aprobar)
`CargoDisputado`	Liquidacion	Bandeja de Administración
`DisputaResuelta`	Liquidacion	Notificaciones

Evento	Lo emite	Reacciona
`DepositoLiberado`	Liquidacion	Notificaciones
`TareaLimpiezaCompletada`	TareaLimpieza	Vehiculo → Disponible
`VehiculoListoParaRenta`	Vehiculo	Disponibilidad de flota, lista de espera

2.6 Diagrama del mapa de dominio



2.7 Lenguaje ubicuo

Un solo vocabulario para negocio, historias, código, base de datos e interfaz. Si el negocio dice “unidad”, el código dice `Vehiculo`. Los nombres van en español, sin acentos.

Término del negocio	En el código	Significado
Reservación	`Reservacion`	Compromiso de una categoría, no de una unidad
Categoría	`CategoriaVehiculo`	Clase de vehículo con tarifa, km incluido y depósito
Unidad	`Vehiculo`	Auto físico, identificado por placa
Contrato de renta	`ContratoRenta`	Acuerdo activo desde la entrega hasta la devolución
Checklist de entrega / devolución	`InspeccionVehiculo` + `TipoInspeccion`	Estado del vehículo con fotos y firma
Cargo adicional	`Cargo`	Cobro por daño, combustible, km o retraso
Liquidación	`Liquidacion`	Cuenta final del contrato: cargos y depósito
Depósito de garantía	`Deposito`	Monto retenido en tarjeta
Upgrade / upgrade de cortesía	`proponerUpgrade` / `otorgarUpgradeCortesia`	Categoría superior, automática o autorizada por Administración
Devolución tardía	`PoliticaCargos.cargoRetraso`	Entrega pasada la tolerancia de 29 min
Extensión de renta	`Extension`	Prolongación del contrato antes de vencer
No-show	`LiberarPorNoShow`	Cliente que no se presenta en 2 h
Rotación	`VehiculoDevuelto` → `VehiculoListoParaRenta`	Tiempo entre la devolución y la nueva disponibilidad

Reglas del lenguaje:

1. Los métodos expresan acciones del negocio (`proponerUpgrade`), no CRUD (`actualizarEstado`).
2. Los eventos van en pasado (`ContratoAbierto`).
3. “Reservación” y “Contrato” nunca son sinónimos.

4. “Categoría” nunca nombra un auto concreto.

3. Service Layer (Application Services)

3.1 Reglas del Service Layer

1. Un método = un caso de uso (con su referencia a la User Story).
2. Cada método abre la transacción, carga el Aggregate, llama un método del dominio, guarda y publica eventos.
3. Una transacción modifica un solo Aggregate. Los demás efectos se disparan por Domain Events (handlers).
4. Verifica que el rol pueda invocar el caso de uso. El umbral de negocio (48 h, 29 min) vive en el dominio.
5. Recibe comandos y devuelve DTOs. Nunca expone entidades.
6. No contiene reglas de negocio: ningún `if` de política.

3.2 Application Services: un método por caso de uso

ReservacionAppService

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`buscarDisponibilidad(cmd)`	US01	Pide la disponibilidad por categoría y arma el DTO	`ServicioDisponibilidad.hayUnidad`
`crearReservacion(cmd)`	US02	Carga Cliente y Categoría → llama `Reservacion.crear` → guarda → publica `ReservacionCreada`	`Reservacion.crear`
`cancelarReservacion(folio, actor)`	US03	Carga → llama `cancelar(ahora, rol, politica)` → guarda → publica `ReservacionCancelada`	`PoliticaCancelacion` (24 h), `Reservacion.cancelar`
`liberarNoShows()`	US04	Job: busca reservaciones vencidas → llama `liberarPorNoShow(ahora)` en cada una → guarda → publica	`PoliticaCancelacion` (2 h), `Reservacion.liberarPorNoShow`
`responderUpgrade(folio, respuesta)`	US09	Carga → llama `aceptarUpgrade` o `rechazarUpgrade(opcion)` → guarda → publica	`Reservacion.aceptarUpgrade`, `Reservacion.rechazarUpgrade`
`otorgarUpgradeCortesia(folio, actor)`	US10	Carga → llama `otorgarUpgradeCortesia(rol)` → guarda	`Reservacion.otorgarUpgradeCortesia` (solo Administración)
`marcarConvertida(folio)`	handler	Carga → llama `marcarConvertida` → guarda	`Reservacion.marcarConvertida`

EntregaAppService

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`iniciarEntrega(folio, documentos)`	US05, US08	`PasarelaPago.verificarTarjeta` → `Cliente.puedeRentar` → `ServicioDisponibilidad.asignarUnidad` → con unidad: `Vehiculo.asignarAEntrega`; sin unidad: `Reservacion.proponerUpgrade` → publica `UpgradeOfrecido`	`Cliente.puedeRentar`, `ServicioDisponibilidad`, `PoliticaUpgrade`

InspeccionAppService

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`registrarInspeccionEntrega(cmd)`	US06	`AlmacenEvidencia.guardar(fotos)` → llama `InspeccionVehiculo.registrar` → guarda	`InspeccionVehiculo.registrar` (sello de tiempo en fotos)
`firmarInspeccionEntrega(id, firma)`	US07	Carga → llama `firmar` → guarda → publica `InspeccionEntregaFirmada`	`InspeccionVehiculo.firmar` (firmada = inmutable)

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`registrarInspeccionDevolucion(cmd)`	US13	Igual que la entrega, con tipo Devolución → publica `InspeccionDevolucionCompletada`	`InspeccionVehiculo.registrar`
`consultarEvidenciaDisputa(liqId, cargoId)`	US17	Solo lectura: arma los checklists y fotos de entrega y devolución	— (consulta, sin reglas)

ContratoAppService

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`abrirContrato(folio, vehiculoId, inspeccionId)`	apertura	`PasarelaPago.retenerDeposito` → llama `ContratoRenta.abrir` → guarda → publica `ContratoAbierto`	`ContratoRenta.abrir`
`extenderRenta(contratoId, horas, actor)`	US11	`ServicioDisponibilidad` → llama `solicitarExtension(..., rol, PoliticaExtension)` → guarda → publica `RentaExtendida` o `ExtensionRequiereAutorizacion`	`PoliticaExtension` (≤ 48 h, ≥ 2 h de anticipación)
`aprobarExtension(contratoId, extId, actor)`	US12	Carga → llama `aprobarExtension(rol)` → guarda → publica `RentaExtendida`	`ContratoRenta.aprobarExtension` (solo Administración)
`procesarDevolucion(contratoId, actor)`	US13 a US16	Carga → llama `registrarDevolucion(ahora)` → guarda → publica `VehiculoDevuelto`	`ContratoRenta.registrarDevolucion`

LiquidacionAppService

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`abrirLiquidacion(contratoId, depositoRef)`	handler	Crea la Liquidacion con el Deposito retenido → guarda	`Liquidacion` (invariantes del depósito)
`calcularCargos(contratoId)`	US14, US15, US16	Handler de `VehiculoDevuelto`. Carga Contrato e Inspecciones → `compararContra` → llama `Liquidacion.calcularCargos(diferencias, condiciones, PoliticaCargos, ahora)` → guarda → publica `CargosCalculados`	`PoliticaCargos` (29 min, 200 km/día, $3 MXN/km, 1/8), `InspeccionVehiculo.compararContra`
`aprobarCargoDanio(liqId, cargoId, monto, actor)`	N5	Carga → llama `aprobarCargoDanio(cargoId, monto, rol)` → guarda	`Liquidacion.aprobarCargoDanio` (el monto lo fija Administración)
`disputarCargo(liqId, cargoId, motivo)`	US17	Carga → llama `disputarCargo` → guarda → publica `CargoDisputado`	`Liquidacion.disputarCargo`
`resolverDisputa(liqId, cargoId, resolucion, actor)`	US17	Carga → llama `resolverDisputa(cargoId, resolucion, rol)` → guarda → publica `DisputaResuelta`	`Liquidacion.resolverDisputa` (solo Administración)
`liberarDepositos()`	US20	Job diario: elegibles → llama `liberarDeposito(ahora, CalendarioHabil)` → `PasarelaPago.liberar` → guarda → publica `DepositoLiberado`	`Liquidacion.liberarDeposito` (5 días hábiles sin incidencias)

FlotaAppService

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`marcarRentado(vehiculoId)`	Handler	Carga → llama `marcarRentado` → guarda	`Vehiculo.marcarRentado`
`registrarDevolucionVehiculo(contratoId)`	Handler	Carga Vehiculo e Inspección de devolución → llama `registrarDevolucion(km, sucursalId)` → guarda	`Vehiculo.registrarDevolucion` (pasa a EnLimpieza)
`marcarDisponible(vehiculoId)`	Handler	Carga → llama `marcarDisponible` → guarda → publica `VehiculoListoParaRenta`	`Vehiculo.marcarDisponible`

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`consultarDisponibilidadFlota(sucursalId)`	N1	Solo lectura sobre el read model	— (consulta, sin reglas)

LimpiezaAppService

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`crearTarea(vehiculoId)`	handler	Llama `TareaLimpieza.crear` → guarda	`TareaLimpieza` (una tarea abierta por devolución)
`marcarVehiculoListo(tareaId, actor)`	US18	Carga → llama `completar(empleadoId, ahora)` → guarda → publica `TareaLimpiezaCompletada`	`TareaLimpieza.completar`

ReporteAppService

Método	US	Orquesta (solo coordinación)	La regla vive en (dominio)
`generarReporteOcupacion(sucursalId, rango)`	US19	Consulta directa a vistas; no carga Aggregates	— (consulta; ver ADR-007)

3.3 Matriz de trazabilidad

US	Caso de uso	Servicio.método
US01	Buscar disponibilidad	`ReservacionAppService.buscarDisponibilidad`
US02	Reservar una categoría	`ReservacionAppService.crearReservacion`
US03	Cancelar una reservación	`ReservacionAppService.cancelarReservacion`
US04	Liberar una reservación por no-show	`ReservacionAppService.liberarNoShows`
US05	Validar al cliente en la entrega	`EntregaAppService.iniciarEntrega`
US06	Registrar el checklist de entrega	`InspeccionAppService.registrarInspeccionEntrega`
US07	Firmar el checklist de entrega	`InspeccionAppService.firmarInspeccionEntrega`
US08	Ofrecer la categoría superior sin costo	`EntregaAppService.iniciarEntrega`
US09	Responder al upgrade	`ReservacionAppService.responderUpgrade`
US10	Autorizar un upgrade de cortesía	`ReservacionAppService.otorgarUpgradeCortesia`
US11	Extender la renta (≤ 48 h)	`ContratoAppService.extenderRenta`
US12	Aprobar una extensión > 48 h	`ContratoAppService.aprobarExtension`
US13	Registrar el checklist de devolución	`InspeccionAppService.registrarInspeccionDevolucion`
US14	Cobrar kilometraje excedente	`LiquidacionAppService.calcularCargos`
US15	Cobrar combustible faltante	`LiquidacionAppService.calcularCargos`
US16	Cobrar devolución tardía	`LiquidacionAppService.calcularCargos`
US17	Resolver la disputa de un cargo por daño	`LiquidacionAppService.disputarCargo`, `resolverDisputa`
US18	Marcar el vehículo como listo	`LimpiezaAppService.marcarVehiculoListo`
US19	Generar el reporte de ocupación	`ReporteAppService.generarReporteOcupacion`
US20	Liberar el depósito de garantía	`LiquidacionAppService.liberarDepositos`

US05 y US08 forman el caso de uso “iniciar entrega”. US14, US15 y US16 son tres reglas del mismo caso de uso “calcular cargos de devolución”. Las reglas están en el dominio, no en el servicio.

3.4 Handlers de eventos

Evento	Handlers (cada uno en su propia transacción)
`ContratoAbierto`	`ReservacionAppService.marcarConvertida`, `FlotaAppService.marcarRentado`, `LiquidacionAppService.abrirLiquidacion`
`VehiculoDevuelto`	`LiquidacionAppService.calcularCargos`, `FlotaAppService.registrarDevolucionVehiculo`, `LimpiezaAppService.crearTarea`
`TareaLimpiezaCompletada`	`FlotaAppService.marcarDisponible`

Los handlers son idempotentes: si el evento llega dos veces, el resultado es el mismo.

3.5 Qué NO hace un Application Service

- No calcula cargos, tolerancias ni topes (eso es `PoliticaCargos`).
- No decide quién autoriza una extensión (eso es `PoliticaExtension`).
- No compara inspecciones (eso es `InspeccionVehiculo.compararContra`).
- No arma consultas de negocio ni SQL.

3.6 Ejemplo de orquestación (pseudocódigo)

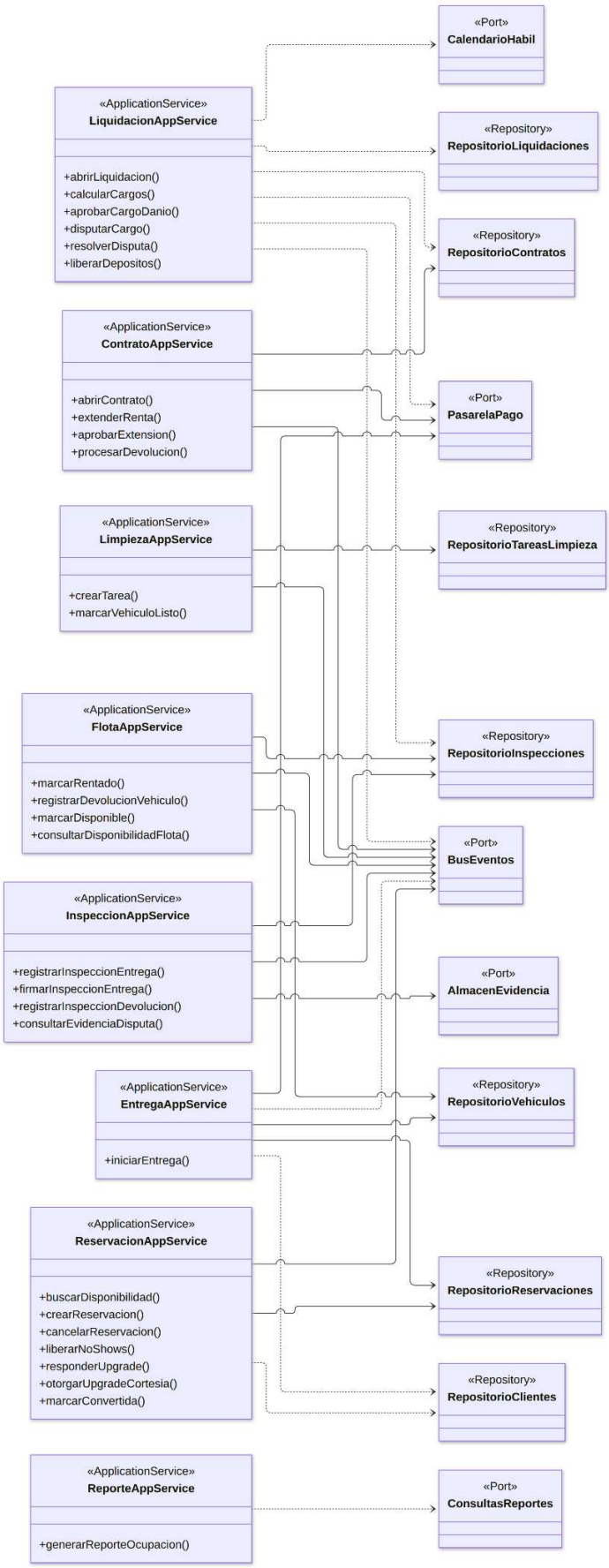
Observa que no hay ningún `if` de regla de negocio: solo permisos, transacción, llamadas al dominio y eventos.

```
class ContratoAppService:
    def procesar_devolucion(self, contrato_id, actor):
        self.permisos.exigir(actor, "procesar_devolucion")  # permiso del caso de uso
        with self.uow:                                      # 1 transacción, 1 aggregate
            contrato = self.contratos.obtener(contrato_id)
            contrato.registrar_devolucion(self.reloj.ahora())  # la regla vive en el dominio
            self.contratos.guardar(contrato)
        self.bus.publicar(contrato.eventos)                 # después del commit

class LiquidacionAppService:
    def calcular_cargos(self, evento):                      # handler de VehiculoDevuelto
        with self.uow:
            contrato = self.contratos.obtener(evento.contrato_id)
            entrega = self.inspecciones.entrega_de(evento.contrato_id)
            devolucion = self.inspecciones.devolucion_de(evento.contrato_id)
            liq = self.liquidaciones.por_contrato(evento.contrato_id)
            liq.calcular_cargos(
                devolucion.comparar_contra(entrega),
                contrato.condiciones(),
                self.politica_cargos,
                self.reloj.ahora(),
            )
            self.liquidaciones.guardar(liq)
        self.bus.publicar(liq.eventos)
```

3.7 Puertos de infraestructura

`RepositorioReservaciones`, `RepositorioClientes`, `RepositorioContratos`, `RepositorioInspecciones`, `RepositorioLiquidaciones`, `RepositorioVehiculos`, `RepositorioTareasLimpieza`, `PasarelaPago`, `AlmacenEvidencia`, `BusEventos`, `CalendarioHabil`, `Reloj`.


«Port»
CalendarioHabil
«ApplicationService»
LiquidacionAppService
+abrirLiquidacion()
+calcularCargos()
+aprobarCargoDanio()
+disputarCargo()
+resolverDisputa()
+liberarDepositos()
«Repository»
RepositorioLiquidaciones
«Repository»
RepositorioContratos
«ApplicationService»
ContratoAppService
+abrirContrato()
+extenderRenta()
+aprobarExtension()
+procesarDevolucion()
«Port»
PasarelaPago
«ApplicationService»
LimpiezaAppService
+crearTarea()
+marcarVehiculoListo()
«Repository»
RepositorioTareasLimpieza
«ApplicationService»
FlotaAppService
+marcarRentado()
+registrarDevolucionVehiculo()
+marcarDisponible()
+consultarDisponibilidadFlota()
«Repository»
RepositorioInspecciones
«Port»
BusEventos
«ApplicationService»
InspeccionAppService
+registrarInspeccionEntrega()
+firmarInspeccionEntrega()
+registrarInspeccionDevolucion()
+consultarEvidenciaDisputa()
«Port»
AlmacenEvidencia
«ApplicationService»
EntregaAppService
+iniciarEntrega()
«Repository»
RepositorioVehiculos
«ApplicationService»
ReservacionAppService
+buscarDisponibilidad()
+crearReservacion()
+cancelarReservacion()
+liberarNoShows()
+responderUpgrade()
+otorgarUpgradeCortesia()
+marcarConvertida()
«Repository»
RepositorioReservaciones
«Repository»
RepositorioClientes
«ApplicationService»
ReporteAppService
+generarReporteOcupacion()
«Port»
ConsultasReportes

3.8 Persistencia y Transacciones

Repository	Necesidad del dominio	Métodos propuestos	Objeto que maneja
RepositorioReservaciones	Ciclo de vida de la reserva y control de no-shows	obtenerPorFolio(), guardar(), buscarVencidasNoShow()	Reservacion
RepositorioContratos	Control de apertura, extensiones y devoluciones	obtenerPorId(), guardar(), buscarActivosPorVehiculo()	ContratoRenta
RepositorioInspecciones	Checklists inmutables de entrega y devolución	guardar(), obtenerEntrega(), obtenerDevolucion()	InspeccionVehiculo
RepositorioLiquidaciones	Cálculo de cargos, depósito retenido y disputas	obtenerPorContrato(), guardar(), buscarElegiblesLiberar()	Liquidacion
RepositorioVehiculos	Disponibilidad física y asignación de flota	obtenerPorId(), guardar(), buscarDisponibles()	Vehiculo

3.9 Separación de Comandos y Consultas - CQRS Básico

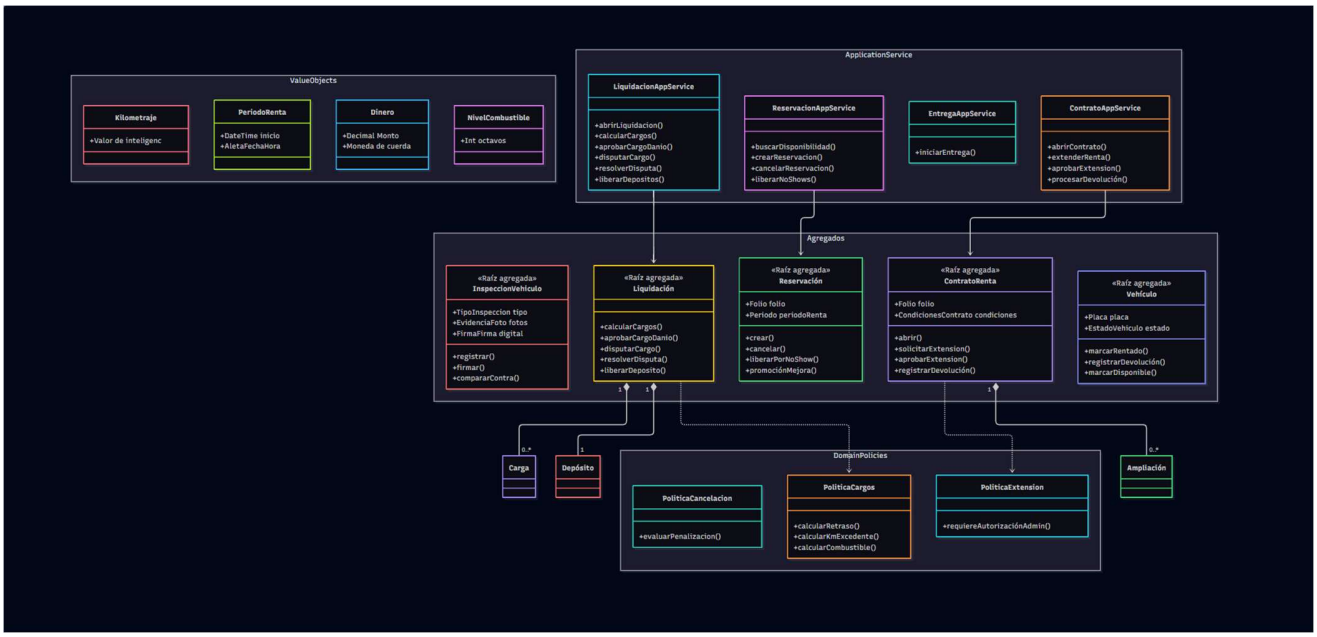
Tipo	Nombre	Entrada	Handler	Resultado / DTO	Justificación
Command	CrearReservacionCmd	clienteId, categoriaId, sucursalId, periodo	ReservacionAppService.crearReservacion	FolioReservacionDTO	Cambia estado y valida reglas del dominio.
Command	ProcesarDevolucionCmd	contratoId, actorId, fechaHora	ContratoAppService.procesarDevolucion	ResultadoDevolucionDTO	Modifica estado del contrato e inicia cálculo financiero.
Query	ConsultarDisponibilidadQuery	sucursalId, rangoFechas	FlotaAppService.consultarDisponibilidad	DisponibilidadViewModel	Consulta de lectura optimizada sin cargar objetos de dominio.
Query	ObtenerReporteOcupacionQuery	sucursalId, rangoFechas	ReporteAppService.generarReporteOcupacion	ReporteOcupacionViewModel	Consulta directa a vistas analíticas de lectura (ADR-002).

3.10 Evaluación y Hallazgos SOLID

Principio	Evidencia en el diseño	Impacto detectado	Cambio / Corrección realizada
S *(SRP)*	LiquidacionAppService procesaba la cuenta final y también generaba reportes analíticos (US19).	Mezcla de responsabilidades transaccionales y de reporte.	**Separación:** Creación de ReporteAppService dedicado a consultas (ADR-002).
O *(OCP)*	Cálculo de cargos (29 min, 200 km/día, 1/8 combustible) en condicionales anidados.	Dificultad para agregar o modificar reglas de negocio sin alterar el servicio.	**Encapsulamiento:** Reglas aisladas en el objeto PoliticaCargos.
L *(LSP)*	Autorización de extensión variaba según el tiempo (≤ 48 h agente vs > 48 h Admin).	Firma inconsistente en los métodos de autorización.	**Polimorfismo:** Interfaz uniforme PoliticaExtension que evalúa roles internamente.
I *(ISP)*	Un repositorio genérico RepositorioRenta expuesto a todas las entidades.	Métodos innecesarios expuestos a componentes que no los requerían.	**Segregación:** Separación en repositorios específicos por Aggregate Root.
D *(DIP)*	Dependencia directa con pasarelas de pago y almacenamiento de imágenes.	Acoplamiento a detalles de infraestructura.	**Inversión:** Introducción de puertos abstractos (PasarelaPago, AlmacenEvidencia).

4. Diagramas UML y ADRs

4.1 Diagrama de Clases DDD Completo



Descripción de Arquitectura y Relaciones del Dominio: El modelo de dominio de *NovaRent* está estructurado bajo los principios de Domain-Driven Design (DDD), aislando la lógica transaccional en *Aggregates* independientes para garantizar la consistencia de las invariantes del negocio:

- **Agregados Principales (Aggregate Roots): Reservacion, ContratoRenta, InspeccionVehiculo, Liquidacion y Vehiculo actúan como las únicas puertas de entrada a los cambios de estado,**

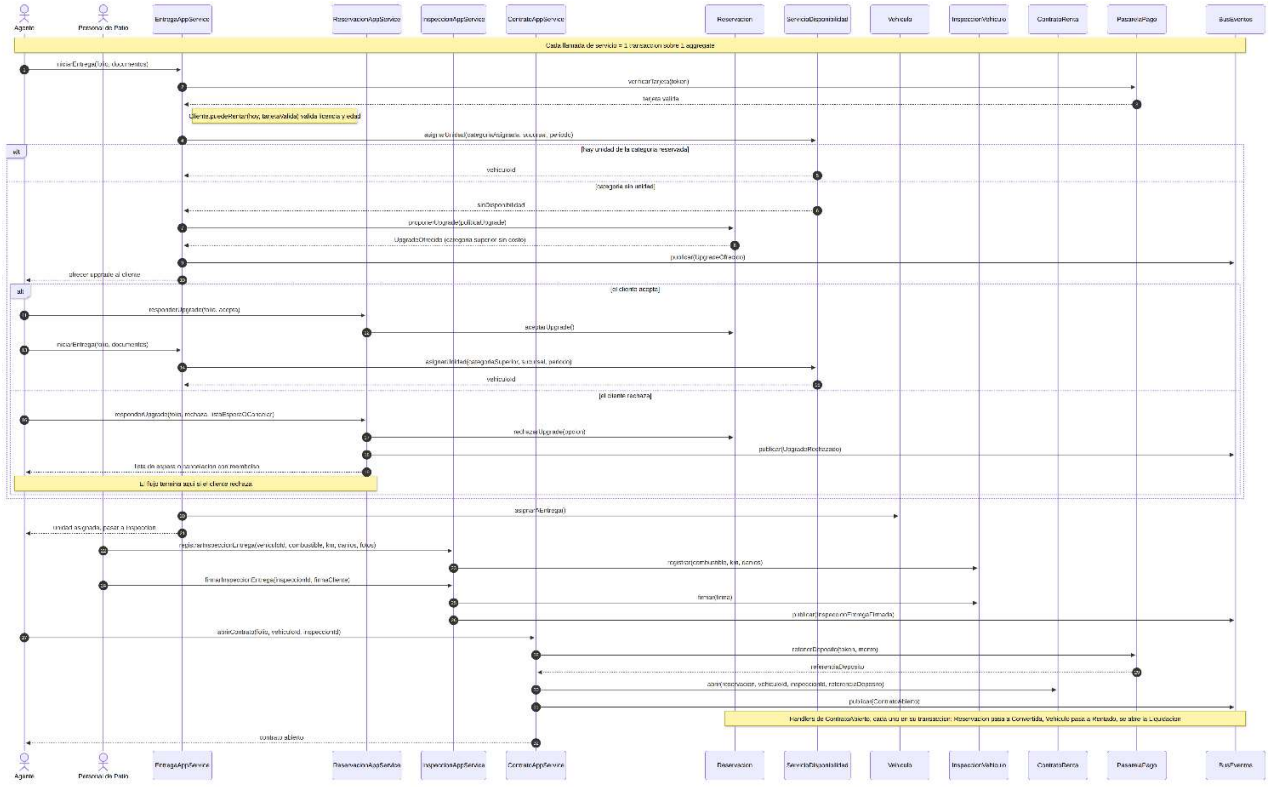
controlando estrictamente sus entidades internas y *Value Objects* (como PeriodoRenta, Dinero, Kilometraje y NivelCombustible).

- **Políticas de Dominio (Domain Policies): Reglas complejas y sujetas a cambios frecuentes (como la PoliticaCancelacion, PoliticaCargos y PoliticaExtension) se encuentran desacopladas en objetos de política puros, permitiendo modificar las reglas de negocio (por ejemplo, tolerancias de 29 minutos o topes de 200 km/día) sin alterar el flujo de los servicios de aplicación.**
- **Comunicación Asíncrona: La integración entre agregados se rige por *Domain Events* (ej. ContratoAbierto, VehiculoDevuelto), asegurando un bajo acoplamiento y consistencia eventual mediante *Event Handlers* ejecutados en transacciones independientes.**

4.2 Diagramas de Secuencia

Diagrama de Secuencia 1: Flujo de Entrega con Asignación/Upgrade (US05/US08)

Descripción del Flujo: El agente inicia la entrega validando al cliente y tarjeta. Si la categoría reservada no tiene unidad disponible, la política de upgrade propone una categoría superior de forma automática.

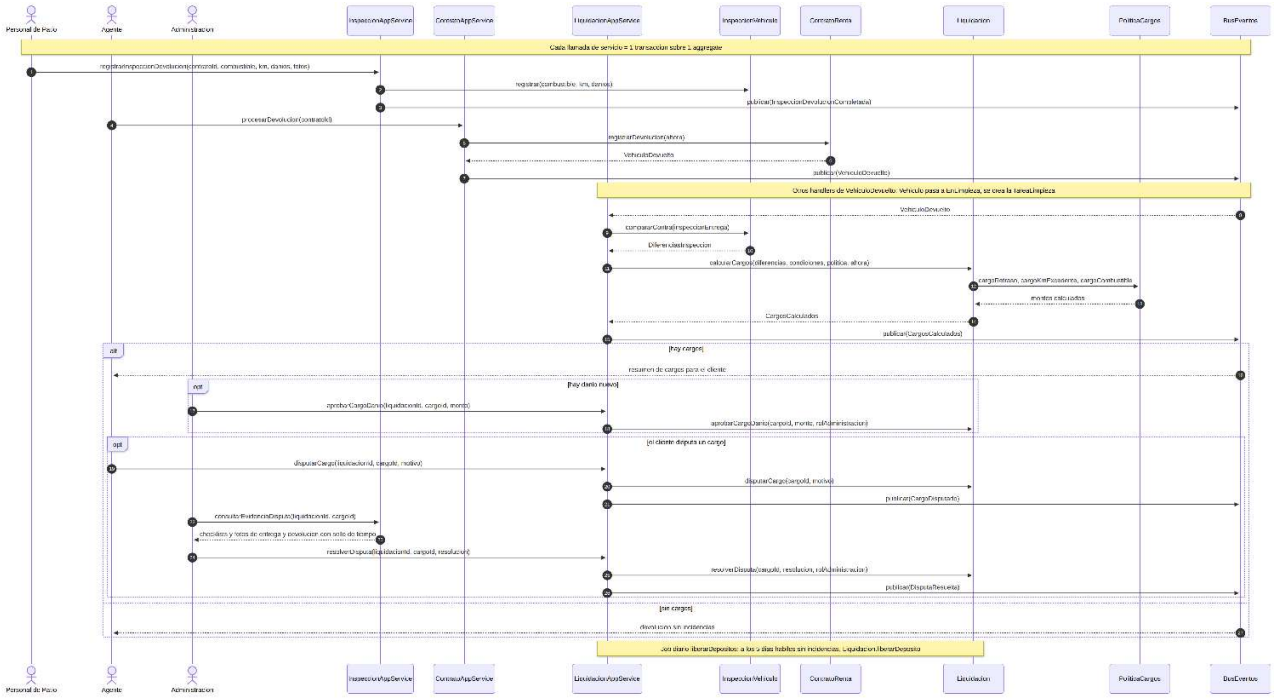


4.2.2 Detalle del Flujo de Devolución y Liquidación (US13 a US16)

Este flujo representa uno de los procesos más robustos del sistema, encargado de gestionar la transición de un vehículo devuelto y el cálculo automatizado de cargos adicionales sin violar los principios de encapsulamiento del dominio.

Paso a paso del flujo técnico:

1. **Invocación del Caso de Uso (ContratoAppService.procesarDevolucion):**
    - El agente de sucursal registra la devolución del contrato proporcionando el identificador y el actor en curso.
    - El servicio de aplicación verifica los permisos correspondientes, abre una transacción (*Unit of Work*) y recupera el ContratoRenta desde su repositorio.
2. **Actualización de Estado en el Dominio:**
    - El agregado ContratoRenta ejecuta el método interno registrar_devolucion(), validando las reglas de negocio temporales y generando los eventos de dominio correspondientes (emitiendo principalmente el evento VehiculoDevuelto).
    - Tras confirmar la transacción, se publican los eventos al bus.
3. **Orquestación Asincrónica (LiquidacionAppService.calcularCargos):**
    - El *Event Handler* intercepta el evento VehiculoDevuelto y desencadena de forma independiente el cálculo financiero.
    - Se recuperan el contrato original, las inspecciones de entrega y de devolución, y el agregado Liquidacion asociado.
4. **Evaluación de Políticas de Negocio (PoliticaCargos):**
    - La inspección de devolución se compara contra la de entrega (devolucion.comparar_contra(entrega)), obteniendo las diferencias exactas en kilometraje, octavos de combustible faltantes y nuevos daños.
    - El agregado Liquidacion delega en la **PoliticaCargos** la aplicación de las reglas paramétricas del negocio:
        - **Retraso:** Se evalúa si el tiempo de entrega supera la tolerancia establecida de 29 minutos.
        - **Kilometraje:** Se calcula el excedente sobre los 200 km incluidos por día a una tarifa fija de $3 MXN/km.
        - **Combustible:** Se valida si la diferencia excede el margen permitido de 1/8 de tanque.
5. **Persistencia y Cierre del Ciclo:**
    - Una vez calculados los montos y cargos correspondientes, la Liquidacion se actualiza y guarda en la base de datos, quedando lista para la aprobación de daños por parte de Administración (si aplica) o la eventual liberación del depósito de garantía a los 5 días hábiles.

6. 

4.3 Registros de Decisiones de Arquitectura (ADRs)

ADR-001: Selección de Domain Model + Service Layer para la Lógica de Negocio

- **Estatus:** Aprobado.
- **Contexto:** El sistema de renta de vehículos NovaRent cuenta con un número elevado de reglas de negocio dinámicas con excepciones (tolerancias de 29 min, topes de kilometraje de 200 km/día, diferencias de combustible de 1/8 de tanque y restricciones de extensión de 48h según el rol del usuario).
- **Decisión:** Implementar un **Domain Model** respaldado por una **Service Layer (Application Services)**. La lógica de negocio vivirá dentro de los *Aggregates* y *Domain Policies*, mientras que los *Application Services* actuarán como coordinadores puros (transacciones, permisos y orquestación).
- **Consecuencias:**
    - *Positivas:* Alta capacidad de realizar pruebas unitarias sin depender de base de datos, código fuertemente alineado con el Lenguaje Ubicuo, bajo acoplamiento con la persistencia.

- *Negativas:* Mayor esfuerzo inicial y curva de aprendizaje para el equipo, necesidad de un mapeo ORM para persistir los Aggregates.

ADR-002: Desviación del Domain Model para la Generación de Reportes (US19)

- **Estatus:** Aprobado.
- **Contexto:** La generación de reportes de ocupación, ingresos y rotación (US19) requiere consultar grandes volúmenes de datos agregados en tiempo real. Cargar estos datos a través de *Aggregates* del *Domain Model* generaría un impacto negativo sustancial en el rendimiento.
- **Decisión:** Eximir los casos de uso de reportes e índices analíticos de pasar por el *Domain Model*. En su lugar, se usará un enfoque de consultas directas a vistas analíticas (estilo *Transaction Script* o patrón de lectura CQRS) a través de ReporteAppService.
- **Consecuencias:**
  - *Positivas:* Consultas eficientes de alto rendimiento directamente a la base de datos sin sobrecarga de memoria por hidratación de objetos del dominio.
  - *Negativas:* Lógica de consulta separada del dominio principal que requiere mantenimiento de las vistas de lectura.

Bitácora de IA

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto de la IA	Decisión del equipo
1	**Elegir patrón**	*"Compara Transaction Script, Table Module y Domain Model con las reglas de NovaRent."*	Identificó que el cambio de políticas favorece **Domain Model**.	Sugirió implementar microservicios y BDs separadas (SQL y NoSQL).	**Corrección:** Usar **Domain Model + Service Layer** en un solo Bounded Context.
2	**Mapa de dominio**	*"Propón entidades, Value Objects y Aggregates con las reglas de NovaRent."*	Separó adecuadamente ContratoRenta, Liquidacion y Value Objects.	Inventó un aggregate Mantenimiento y un sistema de "descuentos".	**Corrección:** Eliminamos componentes inventados fuera del alcance de U2.
3	**Service Layer**	*"Propone firmas de Application Services para los casos US01 a US20."*	Estructuró métodos para ReservacionAppService y ContratoAppService.	Colocó las reglas financieras (if de tolerancias) dentro del servicio.	**Corrección:** Se delegaron las reglas a la policy PoliticaCargos y al dominio.
4	**Revisión SOLID**	*"Analiza el diseño e identifica violaciones SOLID en la Service Layer."*	Detectó que LiquidacionAppService violaba SRP al generar reportes (US19).	Sugirió meter la lógica de lectura dentro del Aggregate Vehiculo.	**Corrección:** Creación de ReporteAppService con consultas directas (ADR-002).
5	**Diagramas UML**	*"Convierte la arquitectura validada en un borrador de código Mermaid."*	Generó la estructura sintáctica básica para diagramas de secuencia.	Usó nombres en inglés (Booking, Car) e inventó herencias de clases.	**Corrección:** Renombramos componentes al **lenguaje ubicuo** en español (Reservacion, Vehiculo).

Repositorio github

https://github.com/cesarmax90/Componentes-logicos